

JSON Place holder API Testing Documentation Project

API Automation Testing using Rest Assured

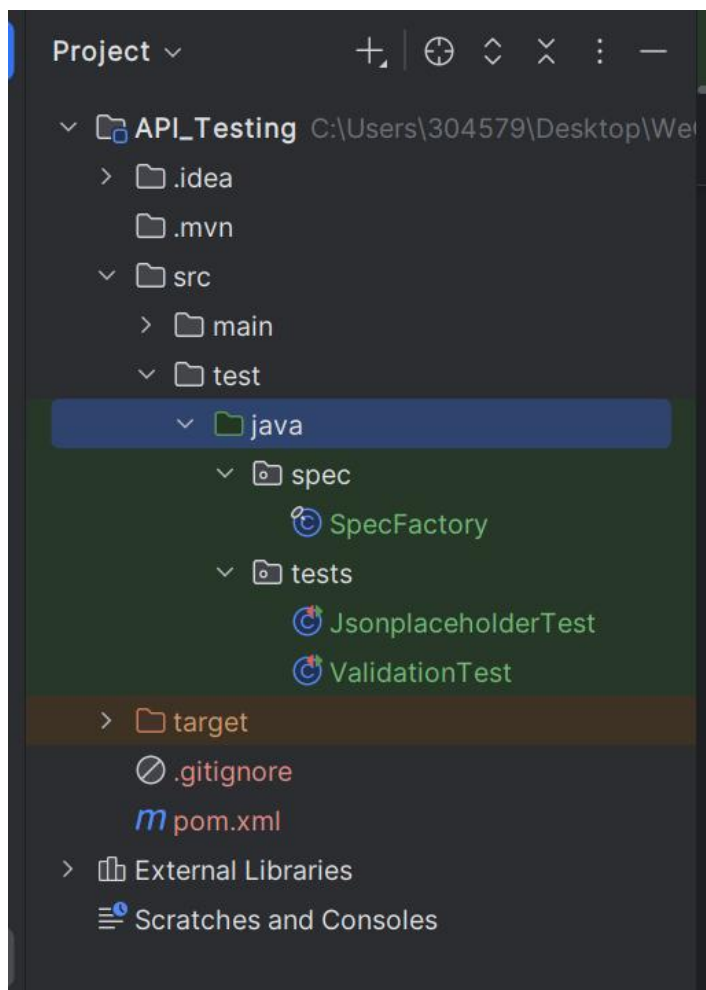
Tools Used

- Java
- Maven
- JUnit 5
- Rest Assured
- IntelliJ IDEA

Base URL: <https://jsonplaceholder.typicode.com>

GitHub Link :

Project Structure



SpecFactory

Contains reusable Request Specification and Response Specifications.

- Base URI
- Headers
- Content Type
- Expected Status Codes

Test Class

Contains API test cases such as:

- Send a GET request to: /posts/1
- Validate:
 - Status Code = 200
 - Id = 1
 - userId is present
 - title is not empty
 - body is not empty
- Expectations:
 - Rest Assured
 - Status code validation
 - JSON Path
 - Assertions
- Validate Headers
- Validate Response Time

Test Scenarios

Test Case	Expected Result
Get	Status Code 200

Validate Response Time

Less than 5 seconds

Validate Headers

Content-Type should be JSON

Code:

SpecFactory.java

```
package spec;

import io.restassured.builder.RequestSpecBuilder;
import io.restassured.builder.ResponseSpecBuilder;
import io.restassured.http.ContentType;
import io.restassured.specification.RequestSpecification;
import io.restassured.specification.ResponseSpecification;

public final class SpecFactory {

    private SpecFactory() {
    }

    public static RequestSpecification requestSpec() {
        return new RequestSpecBuilder()
            .setBaseUrl("https://jsonplaceholder.typicode.com/")
            .setContentType(ContentType.JSON)
            .build();
    }

    public static ResponseSpecification success200() {
        return new ResponseSpecBuilder()
            .expectStatusCode(200)
            .expectContentType(ContentType.JSON)
            .build();
    }
}
```

JsonplaceholderTest.java

```
package tests;

import io.restassured.response.Response;
import org.junit.jupiter.api.Test;
import spec.SpecFactory;

import static io.restassured.RestAssured.given;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertNotNull;

public class JsonplaceholderTest {

    @Test
    void getOne() {
        Response response =
            given()
                .spec(SpecFactory.requestSpec())
                .when()
                .get("/posts/1");

        response.then()
```

```

        .spec(SpecFactory.success200());

    assertEquals(200, response.getStatusCode(), "Status code should be 200");

    int id = response.jsonPath().getInt("id");
    Object userId = response.jsonPath().get("userId");
    String title = response.jsonPath().getString("title");
    String body = response.jsonPath().getString("body");

    assertEquals(1, id, "id should be 1");
    assertNotNull(userId, "userId should be present");
    assertFalse(title == null || title.isBlank(), "title should not be empty");
    assertFalse(body == null || body.isBlank(), "body should not be empty");
}
}

```

ValidationTest.java

```

package tests;

import io.restassured.response.Response;
import org.junit.jupiter.api.Test;
import spec.SpecFactory;

import static io.restassured.RestAssured.given;
import static org.junit.jupiter.api.Assertions.assertTrue;

public class ValidationTest {
    @Test
    void validateHeaders() {
        Response response =
            given()
                .spec(SpecFactory.requestSpec())
                .when()
                .get("/posts");
        response.then()
            .spec(SpecFactory.success200());
        assertTrue(
            response.getHeader("Content-Type")
                .contains("application/json"));
    }

    @Test
    void validateResponseTime() {
        Response response =
            given()
                .spec(SpecFactory.requestSpec())
                .when()
                .get("/posts/1");
        assertTrue(response.time() < 5000);
    }
}

```

Execution

Run all tests:

mvn clean test

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.464 s -- in tests.JsonplaceholderTest
[INFO] Running tests.ValidationTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.464 s -- in tests.ValidationTest
[INFO] Results:
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.073 s
[INFO] Finished at: 2026-06-29T16:17:44+05:30
[INFO] -----
PS C:\Users\304579\Desktop\WeCp\SDET\UIandAPITesting\API_Testing>
```

Run from IntelliJ:

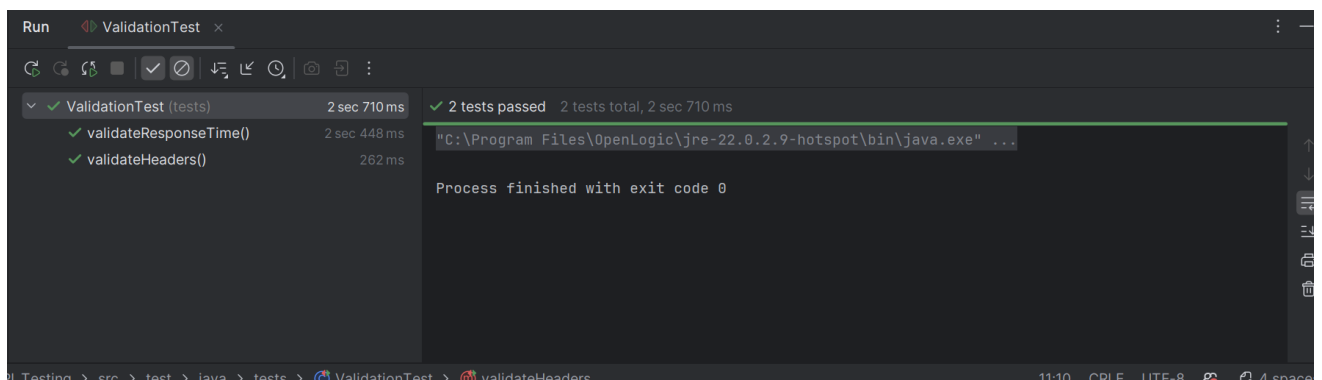
- Right-click Run from IntelliJ •

Click **Run**

1) Get



2) Validate headers and response



Conclusion

The project demonstrates the use of:

- Rest Assured
- Request Specification
- Response Specification
- JUnit 5 Assertions
- API Test Structure